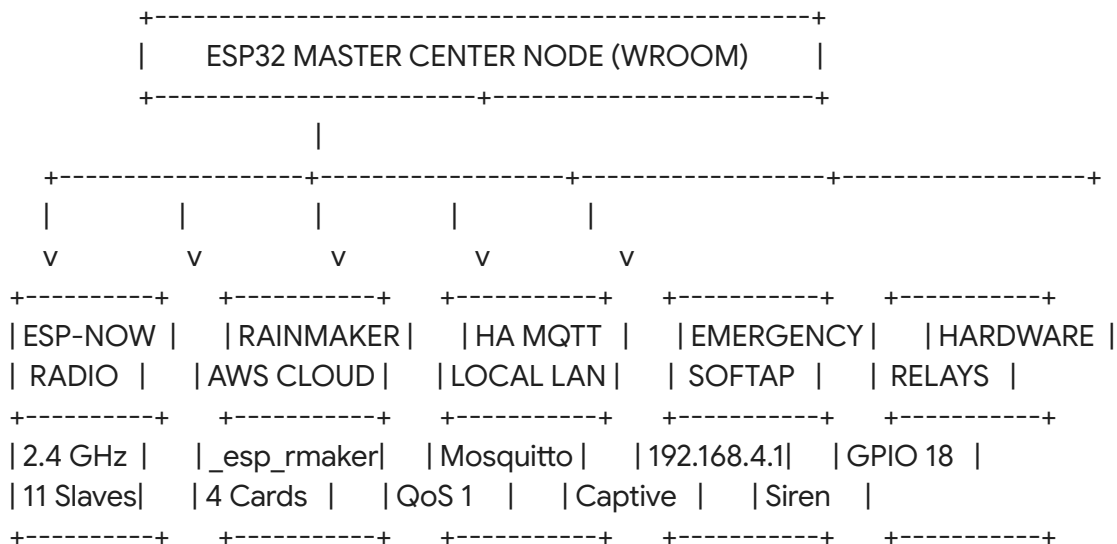


ESP32 Master Center Node: A-to-Z

Engineering Reference & Cheat Sheet

1. System Architecture & Capabilities

This Master Center Node is a **fault-tolerant, multi-protocol IoT hub** built on the ESP32 WROOM platform. It bridges four independent network layers to ensure whole-home automation never fails, even during ISP outages, Wi-Fi router crashes, or AWS server downtime.



Key Operational Modes

- **Normal Mode (Router & Internet Online):**
 - Synchronizes with Expressif RainMaker (AWS Cloud) for mobile app control.
 - Maintains an authenticated MQTT connection to Home Assistant (192.168.0.40:1883) with retain: true state publishing.
 - Transmits and receives real-time ESP-NOW RF packets to/from auxiliary C3 slave nodes.
- **Emergency Mode (Router Offline / Manual Override):**
 - The **60-second Auto-Supervisor** detects router loss and activates an emergency Wi-Fi Access Point (ESP32-EMERGENCY-HUB).
 - A **Captive Portal DNS Server** intercepts client DNS requests and automatically pops up an interactive, dark-themed **Obsidian Web Dashboard** at [\[http://192.168.4.1\]](http://192.168.4.1)(<http://192.168.4.1>).
 - The embedded web dashboard uses **AJAX background polling (1500ms)** to highlight live hardware states without page reloads.

2. Hardware Pinout & Memory Mapping

Pin / Resource	Assignment / Value	Electrical Description	Functional Purpose
GPIO 18	SIREN_RELAY_PIN	Active-HIGH Digital Output	Triggers the industrial security alarm siren relay.
GPIO 0	PROV_BUTTON_PIN	Active-LOW Input (Pull-Up)	BOOT button. Hold for 5 seconds to Factory Reset & Unclaim from AWS.
Virtual 253	bath_dummy_pin	Memory Pointer Only	Isolates REDWAN-BATH-NO DE light override from physical GPIO pull-up traps.
Virtual 254	auto_dummy_pin	Memory Pointer Only	Isolates Bath Radar Auto switch card from physical GPIO 0 pull-up traps.
Virtual 255	ap_dummy_pin	Memory Pointer Only	Isolates Master Hub Controls SoftAP switch from physical GPIO pull-up traps.

3. Communication Protocol Reference

A. ESP-NOW RF Packet Structures (broadcastAddress: FF:FF:FF:FF:FF:FF)

- **Incoming Telemetry (struct_message):**
 - char nodeID[20]: Originating slave identifier (e.g., "REDWAN-BATH-NODE").
 - bool lightActive: Live relay state of the slave (true = ON, false = OFF).
 - uint8_t mode: Slave operational mode (0 = Auto/Radar, 1 = Force ON, 2 = Force OFF).
- **Outgoing Command (cmd_message):**

- uint8_t targetMode: Broadcast instruction sent to slaves (0 = Auto, 1 = Force ON, 2 = Force OFF).

B. Home Assistant MQTT Topic Map

HA Entity Name	MQTT Topic	Direction	Expected Values	Purpose
Siren Relay Switch	home/master_node/siren/set	HA \$rightarrow\$ ESP32	"ON" / "OFF"	Commands physical siren relay on GPIO 18.
Siren State Feedback	home/master_node/siren/state	ESP32 \$rightarrow\$ HA	"ON" / "OFF"	Synchronizes UI switch toggle state in HA.
SoftAP Override Switch	home/master_node/ap_mode/set	HA \$rightarrow\$ ESP32	"ON" / "OFF"	Manually triggers Captive Portal SoftAP.
SoftAP State Feedback	home/master_node/ap_mode/state	ESP32 \$rightarrow\$ HA	"ON" / "OFF"	Synchronizes SoftAP override switch in HA.
Alarm Safety Binary	home/alarms/master_siren	ESP32 \$rightarrow\$ HA	"TRIGGERED" / "SAFE"	Safety binary sensor for alarm automations.
Siren Reason Text	home/alarms/reason	ESP32 \$rightarrow\$ HA	Text String	Diagnostic sensor showing trigger origin.
AP Network Status	home/master_node/ap_mode/status	ESP32 \$rightarrow\$ HA	Text String	Displays live IP or standby status string.

4. Home Assistant Configuration (configuration.yaml)

Add this configuration block under your root mqtt: key. Every entity is bound to the **REDWAN-MASTER-NODE** device registry entry:

```
mqtt:
  switch:
    # --- DEVICE 1: EMERGENCY SIREN CONTROL SWITCH ---
    - name: "Master Emergency Siren Control"
      unique_id: "redwans_master_node_siren_switch_01"
      command_topic: "home/master_node/siren/set"
      state_topic: "home/master_node/siren/state"
      payload_on: "ON"
      payload_off: "OFF"
      icon: "mdi:alarm-light"
      qos: 1
      retain: true
      device:
        identifiers: ["redwans_master_node"]
        name: "REDWAN-MASTER-NODE"
        model: "ESP32 WROOM Center Hub"
        manufacturer: "Expressif / Redwan"

    # --- DEVICE 2: EMERGENCY SOFTAP OVERRIDE SWITCH ---
    - name: "Master Emergency AP Mode Override"
      unique_id: "redwans_master_node_ap_switch_01"
      command_topic: "home/master_node/ap_mode/set"
      state_topic: "home/master_node/ap_mode/state"
      payload_on: "ON"
      payload_off: "OFF"
      icon: "mdi:wifi-access-point"
      qos: 1
      retain: true
      device:
        identifiers: ["redwans_master_node"]
        name: "REDWAN-MASTER-NODE"
        model: "ESP32 WROOM Center Hub"
```

manufacturer: "Expressif / Redwan"

binary_sensor:

--- DEVICE 3: MASTER CENTER NODE (ALARM STATE) ---

- name: "Master Siren Alarm Active"
unique_id: "redwans_master_node_alarm_state_01"
state_topic: "home/alarms/master_siren"
payload_on: "TRIGGERED"
payload_off: "SAFE"
device_class: safety
qos: 1
device:
 identifiers: ["redwans_master_node"]
 name: "REDWAN-MASTER-NODE"
 model: "ESP32 WROOM Center Hub"
 manufacturer: "Expressif / Redwan"

--- DEVICE 4: EMERGENCY AP BROADCASTING STATE ---

- name: "Master AP Mode Active"
unique_id: "redwans_master_node_ap_binary_01"
state_topic: "home/master_node/ap_mode/state"
payload_on: "ON"
payload_off: "OFF"
device_class: connectivity
icon: "mdi:access-point-network"
qos: 1
device:
 identifiers: ["redwans_master_node"]
 name: "REDWAN-MASTER-NODE"
 model: "ESP32 WROOM Center Hub"
 manufacturer: "Expressif / Redwan"

sensor:

--- DEVICE 5: MASTER CENTER NODE (SIREN REASON) ---

- name: "Master Center Node Siren Status"
unique_id: "redwans_master_node_siren_reason_01"
state_topic: "home/alarms/reason"
icon: "mdi:alarm-bell"
qos: 1
device:
 identifiers: ["redwans_master_node"]
 name: "REDWAN-MASTER-NODE"
 model: "ESP32 WROOM Center Hub"

```

    manufacturer: "Expressif / Redwan"

# --- DEVICE 6: EMERGENCY AP IP & NETWORK STATUS ---
- name: "Master AP Network Status"
  unique_id: "redwans_master_node_ap_status_01"
  state_topic: "home/master_node/ap_mode/status"
  icon: "mdi:ip-network"
  qos: 1
  device:
    identifiers: ["redwans_master_node"]
    name: "REDWAN-MASTER-NODE"
    model: "ESP32 WROOM Center Hub"
    manufacturer: "Expressif / Redwan"

```

5. Complete Production Firmware (master-node.ino)

```

#include <WiFi.h>
#include <esp_now.h>
#include <WebServer.h> // Native lightweight HTTP server
#include <DNSServer.h> // Captive Portal Auto-Popup DNS
#include <PubSubClient.h> // Standard MQTT Client for Home Assistant
#include "RMaker.h"
#include "WiFiProv.h"
#include "RMakerQR.h"

// =====
// 1. PIN ASSIGNMENTS & HARDWARE LOGIC
// =====
#define SIREN_RELAY_PIN 18 // Industrial Siren Relay (Active-HIGH)
#define PROV_BUTTON_PIN 0 // BOOT button: Hold 5s for Factory Reset & Unclaim

// =====
// 2. CONSTANTS, VARIABLES & TIMERS
// =====
const char *service_name = "CENTER_NODE";
const char *pop = "abcd2005-"; // BLE Provisioning Proof-of-Possession PIN

const char *softap_ssid = "ESP32-EMERGENCY-HUB";
const char *softap_pass = "abcd2005-"; // Emergency Access Point password

```

```

// *** HOME ASSISTANT MQTT BROKER CREDENTIALS ***
const char *MQTT_SERVER = "192.168.0.40"; // Raspberry Pi HA Broker IP
const int MQTT_PORT = 1883;
const char *MQTT_USER = "redwanmqtt";
const char *MQTT_PASS = "abcd2005-";

static uint8_t siren_pin = SIREN_RELAY_PIN;
static uint8_t bath_dummy_pin = 253; // Virtual dummy pin
static uint8_t auto_dummy_pin = 254; // Virtual dummy pin for Radar Auto Card
static uint8_t ap_dummy_pin = 255; // Virtual dummy pin

// Device Cards: Siren | Bath Light | Bath Auto Toggle | Emergency AP Card
static Switch *mySirenSwitch = NULL;
static Switch *myBathNodeDevice = NULL;
static Switch *myBathAutoCard = NULL;
static Switch *myApControlCard = NULL;

// Custom auxiliary text status parameters
static Param *bathStatusParam = NULL;
static Param *alarmStatusParam = NULL;
static Param *apStatusParam = NULL;

bool emergencyAlarmActive = false;
bool bathLightForceState = false;
bool bathRadarAutoState = true;
bool isEmergencyApActive = false;
bool manualApOverride = false;

unsigned long lastWifiCheckTime = 0;
unsigned long lastMqttRetryTime = 0;
const unsigned long WIFI_CHECK_INTERVAL_MS = 60000;
const unsigned long MQTT_RETRY_INTERVAL_MS = 5000;

// Broadcast MAC for transmitting override commands to C3 Slaves
uint8_t broadcastAddress[] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };

typedef struct struct_message {
    char nodeID[20];
    bool lightActive;
    uint8_t mode; // 0=Auto, 1=ForceON, 2=ForceOFF
} struct_message;
struct_message incomingSlaveData;

```

```

typedef struct cmd_message {
    uint8_t targetMode; // 0=Auto, 1=ForceON, 2=ForceOFF
} cmd_message;
cmd_message outgoingCmd;

const byte DNS_PORT = 53;
DNSServer dnsServer;      // Captive Portal DNS Server
    emergencyServer 80 ; // Lightweight Emergency HTTP Server on Port 80

WiFiClient espClient;
    mqttClient      ;

// Function prototypes
void startEmergencyAP ;
void stopEmergencyAP ;
void setupEmergencyEndpoints ;
void publishHAState ;
void reconnectMQTT ;
void mqttCallback char                unsigned int      ;

// =====
// 3. OFFICIAL PROVISIONING EVENT HANDLER
// =====
void sysProvEvent arduino_event_t      {
    switch (sys_event->event_id) {
        case ARDUINO_EVENT_PROV_START:
            Serial.printf("\n[PROV] BLE Provisioning Started! Name: \"%s\" | PoP: \"%s\"\n", service_name,
pop);
            printQR(service_name, pop, "ble");
            break;
        case ARDUINO_EVENT_WIFI_STA_CONNECTED:
            Serial.println("\n[WIFI] Connected to Router!");
            break;
        case ARDUINO_EVENT_PROV_END:
            Serial.println("\n[PROV] Provisioning Closed! BLE memory released.");
            break;
        default:
            break;
    }
}

// =====

```

```

// 4. HOME ASSISTANT MQTT STATE PUBLISHER
// =====
void publishHAState {
    if (!mqttClient.connected()) return;
    mqttClient.publish("home/master_node/siren/state", emergencyAlarmActive ? "ON" : "OFF",
true);
    mqttClient.publish("home/master_node/ap_mode/state", isEmergencyApActive ? "ON" : "OFF",
true);
    mqttClient.publish("home/alarms/master_siren", emergencyAlarmActive ? "TRIGGERED" :
"SAFE", true);
    mqttClient.publish("home/alarms/reason", emergencyAlarmActive ? "Manual Alarm Active" :
"System Armed - All Safe", true);
    mqttClient.publish("home/master_node/ap_mode/status", isEmergencyApActive ? "AP
ACTIVE: 192.168.4.1" : "AP STANDBY (Router Online)", true);
}

// =====
// 5. HOME ASSISTANT MQTT COMMAND LISTENER
// =====
void mqttCallback(char unsigned int {
    String cmd = "";
    for (unsigned int i = 0; i < length; i++) {
        cmd += (char)payload[i];
    }
    Serial.printf("[MQTT RX] Topic: %s | Cmd: %s\n", topic, cmd.c_str());

    // A. Emergency Siren Command from Home Assistant
    if (strcmp(topic, "home/master_node/siren/set") == 0) {
        emergencyAlarmActive = (cmd == "ON");
        digitalWrite(SIREN_RELAY_PIN, emergencyAlarmActive ? HIGH : LOW);
        if (mySirenSwitch) mySirenSwitch->updateAndReportParam("Power",
emergencyAlarmActive);
        if (!emergencyAlarmActive && alarmStatusParam) {
            alarmStatusParam->updateAndReport(value("SYSTEM SAFE - ALL CLEAR"));
        }
        publishHAState();
    }

    // B. Emergency AP Override Command from Home Assistant
    else if (strcmp(topic, "home/master_node/ap_mode/set") == 0) {
        manualApOverride = (cmd == "ON");
        if (myApControlCard) myApControlCard->updateAndReportParam("Power",
manualApOverride);
        if (manualApOverride && !isEmergencyApActive) {

```

```

        startEmergencyAP();
    } else if (!manualApOverride && WiFi.status() == WL_CONNECTED &&
isEmergencyApActive) {
        stopEmergencyAP();
    }
    publishHAState();
}
}

void reconnectMQTT {
    if (mqttClient.connected() || (millis() - lastMqttRetryTime < MQTT_RETRY_INTERVAL_MS))
return;
    lastMqttRetryTime = millis();

    Serial.print("[MQTT] Connecting to Home Assistant Broker (192.168.0.40)...");
    if (mqttClient.connect("REDWAN_MASTER_NODE_ESP32", MQTT_USER, MQTT_PASS)) {
        Serial.println("CONNECTED!");
        mqttClient.subscribe("home/master_node/siren/set");
        mqttClient.subscribe("home/master_node/ap_mode/set");
        publishHAState();
    } else {
        Serial.printf("FAILED (rc=%d). Will retry in 5s...\n", mqttClient.state());
    }
}

// =====
// 6. ESP RAINMAKER CLOUD CALLBACK
// =====
void write_callback(const param_val_t val, void
write_ctx_t ctx) {
    const char *device_name = device->getDeviceName();
    const char *param_name = param->getParamName();

    // A. EMERGENCY SIREN CARD
    if (strcmp(device_name, "Emergency Siren") == 0 && strcmp(param_name, "Power") == 0) {
        emergencyAlarmActive = val.val.b;
        digitalWrite(SIREN_RELAY_PIN, emergencyAlarmActive ? HIGH : LOW);
        Serial.printf("[RAINMAKER] Siren manually toggled to: %s\n", emergencyAlarmActive ? "ON" :
"OFF");
        param->updateAndReport(val);
        if (!emergencyAlarmActive && alarmStatusParam) {
            alarmStatusParam->updateAndReport(value("SYSTEM SAFE - ALL CLEAR"));
        }
    }
}

```

```

    publishHAState();
}

// B. BATHROOM LIGHT FORCE CARD
if (strcmp(device_name, "REDWAN-BATH-NODE") == 0 && strcmp(param_name, "Power") == 0) {
    bathLightForceState = val.val.b;
    bathRadarAutoState = false; // Forcing light disables auto mode
    Serial.printf("[RAINMAKER] Bath Light override toggled to: %s\n", bathLightForceState ? "ON" : "OFF");

    outgoingCmd.targetMode = bathLightForceState ? 1 : 2; // 1 = FORCE_ON, 2 = FORCE_OFF
    esp_now_send(broadcastAddress, (uint8_t *)&outgoingCmd, sizeof(outgoingCmd));

    if (myBathAutoCard) myBathAutoCard->updateAndReportParam("Power", false);
    if (bathStatusParam) {
        bathStatusParam->updateAndReport(value(bathLightForceState ? "MANUAL - Light ON" : "MANUAL - Light OFF"));
    }
    param->updateAndReport(val);
}

// C. BATHROOM RADAR AUTO TOGGLE CARD (Dedicated Switch Card = True UI Toggle!)
if (strcmp(device_name, "Bath Radar Auto") == 0 && strcmp(param_name, "Power") == 0) {
    bathRadarAutoState = val.val.b;
    if (bathRadarAutoState) {
        Serial.println("[RAINMAKER] Radar Auto Enabled -> Restoring AUTO(0) to Slaves");
        outgoingCmd.targetMode = 0; // AUTO_MODE
        esp_now_send(broadcastAddress, (uint8_t *)&outgoingCmd, sizeof(outgoingCmd));
        if (bathStatusParam) bathStatusParam->updateAndReport(value("AUTO - Radar Active"));
    }
    param->updateAndReport(val);
}

// D. MASTER HUB AP OVERRIDE CARD
if (strcmp(device_name, "Master Hub Controls") == 0 && strcmp(param_name, "Power") == 0) {
    manualApOverride = val.val.b;
    Serial.printf("[RAINMAKER] Manual Emergency AP Override: %s\n", manualApOverride ? "ON" : "OFF");

    if (manualApOverride && !isEmergencyApActive) {
        startEmergencyAP();
    }
}

```

```

    } else if (!manualApOverride && WiFi.status() == WL_CONNECTED &&
isEmergencyApActive) {
        stopEmergencyAP();
    }
    param->updateAndReport(val);
    publishHAState();
}
}

// =====
// 7. ESP-NOW SLAVE RECEIVER CALLBACK
// =====
void onESPNowDataRecv(const          const uint8_t          int
{
    memcpy(&incomingSlaveData, incomingData, sizeof(incomingSlaveData));
    Serial.printf("[ESP-NOW SLAVE] From %s -> State: %s | Mode: %d\n",
        incomingSlaveData.nodeID,
        incomingSlaveData.lightActive ? "ON" : "OFF",
        incomingSlaveData.mode);

    if (strcmp(incomingSlaveData.nodeID, "REDWAN-BATH-NODE") == 0) {
        bathLightForceState = incomingSlaveData.lightActive;
        bathRadarAutoState = (incomingSlaveData.mode == 0);

        if (myBathNodeDevice) myBathNodeDevice->updateAndReportParam("Power",
bathLightForceState);
        if (myBathAutoCard) myBathAutoCard->updateAndReportParam("Power",
bathRadarAutoState);

        if (bathStatusParam) {
            String stat = (incomingSlaveData.mode == 0) ? "AUTO - " : "MANUAL - ";
            stat += incomingSlaveData.lightActive ? "Light ON" : "Light OFF";
            bathStatusParam->updateAndReport(value(stat.c_str()));
        }
    }
}

// =====
// 8. INTERACTIVE OBSIDIAN HTML DASHBOARD (REAL-TIME POLLING)
// =====
const char EMERGENCY_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>ESP32 Emergency Hub</title>
<style>
  body { background: #111115; color: #e0e0e0; font-family: 'Segoe UI', Tahoma, Geneva, Verdana,
sans-serif; margin: 0; padding: 20px; }
  .container { max-width: 440px; margin: 0 auto; }
  h2 { color: #bb86fc; border-bottom: 1px solid #2a2a32; padding-bottom: 8px; font-size: 14px;
text-transform: uppercase; letter-spacing: 1.5px; margin-top: 24px; }
  .card { background: #191920; border: 1px solid #282832; border-radius: 14px; padding: 18px;
margin-bottom: 16px; box-shadow: 0 8px 16px rgba(0,0,0,0.4); }
  .row { display: flex; justify-content: space-between; align-items: center; margin-bottom: 12px; }
  .row:last-child { margin-bottom: 0; }
  .label { font-weight: 600; font-size: 15px; color: #f0f0f0; }
  .sub { font-size: 12px; color: #888896; margin-top: 2px; }
  .btn-group { display: flex; gap: 8px; }
  .btn { background: #23232d; color: #9999a6; border: 1px solid #333340; padding: 10px 16px;
border-radius: 8px; font-weight: 700; cursor: pointer; transition: all 0.2s ease; flex: 1; text-align:
center; }
  .btn:hover { border-color: #555566; color: #ffffff; }
  .btn.active-red { background: #ff1744 !important; color: #ffffff !important; border-color: #ff526d
!important; box-shadow: 0 0 14px rgba(255, 23, 68, 0.4); }
  .btn.active-purple { background: #7c4dff !important; color: #ffffff !important; border-color:
#9e7eff !important; box-shadow: 0 0 14px rgba(124, 77, 255, 0.4); }
  .btn.active-cyan { background: #00e5ff !important; color: #111115 !important; border-color:
#6effff !important; box-shadow: 0 0 14px rgba(0, 229, 255, 0.4); }
</style>
</head>
<body>
<div class="container">
  <h2>— Security Alarm —</h2>
  <div class="card">
    <div class="row">
      <div>
        <div class="label">Emergency Siren</div>
        <div class="sub">Industrial Siren Relay</div>
      </div>
      <div class="btn-group" style="width: 150px;">
        <button id="siren-on" class="btn" onclick="sendCommand('siren', 'on')>ON</button>
        <button id="siren-off" class="btn" onclick="sendCommand('siren', 'off')>OFF</button>
      </div>
    </div>
  </div>

  <h2>— REDWAN-BATH-NODE —</h2>
  <div class="card">
    <div class="row" style="margin-bottom: 18px;">
      <div>

```

```

    <div class="label">Light Override</div>
    <div class="sub">Direct Relay Control</div>
  </div>
  <div class="btn-group" style="width: 150px;">
    <button id="light-on" class="btn" onclick="sendCommand('bath_light', 'on')">ON</button>
    <button id="light-off" class="btn" onclick="sendCommand('bath_light', 'off')">OFF</button>
  </div>
</div>
<div class="row">
  <div>
    <div class="label">Radar Auto Mode</div>
    <div class="sub">HLK-LD2410B Sensor</div>
  </div>
  <div class="btn-group" style="width: 180px;">
    <button id="auto-on" class="btn" onclick="sendCommand('bath_auto', 'on')">AUTO</button>
    <button id="auto-off" class="btn" onclick="sendCommand('bath_auto',
'off')">MANUAL</button>
  </div>
</div>
</div>
</div>
</div>

<script>
function updateUI(state) {
  document.getElementById('siren-on').className = state.siren ? 'btn active-red' : 'btn';
  document.getElementById('siren-off').className = !state.siren ? 'btn active-red' : 'btn';

  document.getElementById('light-on').className = state.light ? 'btn active-purple' : 'btn';
  document.getElementById('light-off').className = !state.light ? 'btn active-purple' : 'btn';

  document.getElementById('auto-on').className = state.auto ? 'btn active-cyan' : 'btn';
  document.getElementById('auto-off').className = !state.auto ? 'btn active-cyan' : 'btn';
}

function sendCommand(device, action) {
  fetch(`/cmd?device=${device}&action=${action}`)
    .then(res => res.json())
    .then(data => updateUI(data))
    .catch(err => console.error("Error:", err));
}

function fetchStatus() {
  fetch('/status')
    .then(res => res.json())
    .then(data => updateUI(data))
    .catch(err => console.log("Polling..."));
}

```

```

// POLL EVERY 1.5 SECONDS: Updates UI automatically when radar detects movement!
setInterval(fetchStatus, 1500);
fetchStatus();
</script>
</body>
</html>
)rawliteral";

```

```

void setupEmergencyEndpoints {
  emergencyServer.on("/", []() {
    emergencyServer.send(200, "text/html", EMERGENCY_HTML);
  });

```

```

  emergencyServer.on("/status", []() {
    String json = "{";
    json += "\"siren\":" + String(emergencyAlarmActive ? "true" : "false") + ",";
    json += "\"light\":" + String(bathLightForceState ? "true" : "false") + ",";
    json += "\"auto\":" + String(bathRadarAutoState ? "true" : "false");
    json += "}";
    emergencyServer.send(200, "application/json", json);
  });

```

```

  emergencyServer.on("/cmd", []() {
    String dev = emergencyServer.arg("device");
    String act = emergencyServer.arg("action");

```

```

    if (dev == "siren") {
      emergencyAlarmActive = (act == "on");
      digitalWrite(SIREN_RELAY_PIN, emergencyAlarmActive ? HIGH : LOW);
      publishHState();
    } else if (dev == "bath_light") {
      bathLightForceState = (act == "on");
      bathRadarAutoState = false;
      outgoingCmd.targetMode = bathLightForceState ? 1 : 2;
      esp_now_send(broadcastAddress, (uint8_t *)&outgoingCmd, sizeof(outgoingCmd));
    } else if (dev == "bath_auto") {
      bathRadarAutoState = (act == "on");
      if (bathRadarAutoState) {
        outgoingCmd.targetMode = 0;
        esp_now_send(broadcastAddress, (uint8_t *)&outgoingCmd, sizeof(outgoingCmd));
      }
    }
  }
}

```

```

String json = "{";
json += "\"siren\":" + String(emergencyAlarmActive ? "true" : "false") + ",";
json += "\"light\":" + String(bathLightForceState ? "true" : "false") + ",";
json += "\"auto\":" + String(bathRadarAutoState ? "true" : "false");
json += "}";
emergencyServer.send(200, "application/json", json);
});

emergencyServer.onNotFound([]() {
  emergencyServer.sendHeader("Location", "http://192.168.4.1/", true);
  emergencyServer.send(302, "text/plain", "");
});
}

void startEmergencyAP {
  if (isEmergencyApActive) return;
  Serial.println("[AP-SUPERVISOR] Starting Emergency Access Point: ESP32-EMERGENCY-HUB ...");

  WiFi.softAP(softap_ssid, softap_pass);
  dnsServer.start(DNS_PORT, "*", WiFi.softAPIP());
  setupEmergencyEndpoints();
  emergencyServer.begin();

  isEmergencyApActive = true;
  if (apStatusParam) apStatusParam->updateAndReport(value("AP ACTIVE: 192.168.4.1"));
  publishHAState();
  Serial.println("[AP-SUPERVISOR] Captive Portal Web Dashboard LIVE at http://192.168.4.1 !");
}

void stopEmergencyAP {
  if (!isEmergencyApActive) return;
  Serial.println("[AP-SUPERVISOR] Router restored. Shutting down Emergency AP...");
  dnsServer.stop();
  emergencyServer.close();
  WiFi.softAPdisconnect(true);
  isEmergencyApActive = false;
  if (apStatusParam) apStatusParam->updateAndReport(value("AP STANDBY (Router Online)"));
  publishHAState();
}

// =====
// 9. SETUP ROUTINE

```

```

// =====
void setup {
  Serial.begin(115200);
  pinMode(SIREN_RELAY_PIN, OUTPUT);
  digitalWrite(SIREN_RELAY_PIN, LOW);
  pinMode(PROV_BUTTON_PIN, INPUT_PULLUP);

  WiFi.mode(WIFI_AP_STA);

  // --- INITIALIZE ESP RAINMAKER NODE & 4 CARDS ---
  Node my_node = RMaker.initNode("ESP32-Center-Node", "HomeAutomation");

  // CARD 1: EMERGENCY SIREN
  mySirenSwitch = new Switch("Emergency Siren", &siren_pin);
  mySirenSwitch->addCb(write_callback);
  alarmStatusParam = new Param("Security Status", "esp.param.name", value("SYSTEM SAFE"),
PROP_FLAG_READ | PROP_FLAG_WRITE);
  mySirenSwitch->addParam(*alarmStatusParam);
  my_node.addDevice(*mySirenSwitch);

  // CARD 2: REDWAN-BATH-NODE LIGHT FORCE
  myBathNodeDevice = new Switch("REDWAN-BATH-NODE", &bath_dummy_pin);
  myBathNodeDevice->addCb(write_callback);
  bathStatusParam = new Param("Live Status", "esp.param.name", value("AUTO - Standby"),
PROP_FLAG_READ);
  myBathNodeDevice->addParam(*bathStatusParam);
  my_node.addDevice(*myBathNodeDevice);

  // CARD 3: BATH RADAR AUTO (Dedicated Card = physical toggle in app!)
  myBathAutoCard = new Switch("Bath Radar Auto", &auto_dummy_pin);
  myBathAutoCard->addCb(write_callback);
  my_node.addDevice(*myBathAutoCard);

  // CARD 4: MASTER HUB CONTROLS (SOFTAP OVERRIDE)
  myApControlCard = new Switch("Master Hub Controls", &ap_dummy_pin);
  myApControlCard->addCb(write_callback);
  apStatusParam = new Param("Fallback Network", "esp.param.name", value("AP STANDBY
(Router Online)"), PROP_FLAG_READ);
  myApControlCard->addParam(*apStatusParam);
  my_node.addDevice(*myApControlCard);

  RMaker.enableOTA(OTA_USING_TOPICS, "Official_Release");
  RMaker.enableTZService();

```

```

RMaker.enableSchedule();
RMaker.start();

// --- PROVISIONING, ESP-NOW & MQTT INITIALIZATION ---
WiFi.onEvent(sysProvEvent);
WiFiProv.beginProvision(NETWORK_PROV_SCHEME_BLE,
NETWORK_PROV_SCHEME_HANDLER_FREE_BTDM, NETWORK_PROV_SECURITY_1, pop,
service_name);

if (esp_now_init() == ESP_OK) {
  esp_now_register_recv_cb(onESPNowDataRecv);
  esp_now_peer_info_t peerInfo = {};
  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;
  esp_now_add_peer(&peerInfo);
}

// Setup MQTT broker connection parameters
mqttClient.setServer(MQTT_SERVER, MQTT_PORT);
mqttClient.setCallback(mqttCallback);

// --- CALIBRATION FIX: FORCE OFF BOOT STATE ---
mySirenSwitch->updateAndReportParam("Power", false);
myApControlCard->updateAndReportParam("Power", false);
myBathNodeDevice->updateAndReportParam("Power", false);
myBathAutoCard->updateAndReportParam("Power", true);

lastWifiCheckTime = millis();

Serial.println("[BOOT] Master Center Node (RainMaker + HA MQTT + Real-Time SoftAP) Online!");
}

// =====
// 10. MAIN PRODUCTION LOOP
// =====
void loop {
  // 1. BOOT Button Hold for 5s -> Factory Reset
  if (digitalRead(PROV_BUTTON_PIN) == LOW) {
    unsigned long pressStart = millis();
    while (digitalRead(PROV_BUTTON_PIN) == LOW) {
      if (millis() - pressStart > 5000) {
        Serial.println("[RESET] Restoring Factory Settings & Unclaiming...");
      }
    }
  }
}

```

```

    RMakerFactoryReset(0);
  }
  delay(50);
}
}

// 2. KEEP HOME ASSISTANT MQTT ALIVE (When Station Wi-Fi is connected)
if (WiFi.status() == WL_CONNECTED) {
  if (!mqttClient.connected()) {
    reconnectMQTT();
  }
  mqttClient.loop();
}

// 3. EMERGENCY AP SUPERVISOR (Checks Router state every 60s)
if (millis() - lastWifiCheckTime >= WIFI_CHECK_INTERVAL_MS) {
  lastWifiCheckTime = millis();

  // Auto-Enable AP if router drops
  if (WiFi.status() != WL_CONNECTED && !isEmergencyApActive) {
    Serial.println("[AP-SUPERVISOR] Router Offline Detected! Broadcasting Captive AP...");
    startEmergencyAP();
  }

  // Auto-Disable AP when router recovers
  else if (WiFi.status() == WL_CONNECTED && !manualApOverride &&
isEmergencyApActive) {
    stopEmergencyAP();
  }
}

// 4. Handle Emergency Web & Captive Portal Requests
if (isEmergencyApActive) {
  dnsServer.processNextRequest();
  emergencyServer.handleClient();
}

delay(10); // Prevents FreeRTOS watchdog starvation
}

```

6. How to Save This as a Clean Reference PDF

To export this Markdown document into a clean PDF document:

1. **In VS Code / Cursor / Markdown Editors:** Open the Markdown Preview tab (Ctrl + Shift

- + V) -> Right-click inside the preview -> Select **Print...** -> Choose **Save as PDF**.
2. **In Chrome / Edge Browser:** Press **Ctrl + P** -> Set **Destination** to **Save as PDF** -> Uncheck *Headers and footers* -> Set **Margins** to *Default* -> Click **Save**.